Serving a Hybrid Linear-Attention MoE Model in a Minimal Inference Engine: Quantization, Decode Bottlenecks, and Multi-GPU Topology

Salem Essid
SUP'COM, University of Carthage, Tunisia
Work done during an internship at Telcotec
salem.essid@supcom.tn

***Abstract*—Hybrid language models interleave linear-attention layers, which carry a fixed-size recurrent state, with full-attention layers that need a growing KV cache, and often route every token through a sparse Mixture-of-Experts (MoE). We describe qLLM, an extension of the minimal nano-vLLM engine that serves Qwen3.5-35B-A3B (30 Gated DeltaNet layers, 10 grouped-query attention layers, 256 experts per layer), and measure what limited its decode throughput at each stage of optimization. Three findings stand out. (1) Grouped INT8 weight-only (W8A16) quantization of the experts cut weight memory 1.84× with no detected GSM8K regression, but on its own made decoding 2.5× *slower*; the net 3.2× gain over BF16 appeared only once a fused MoE kernel and CUDA-graph capture were added. (2) On a single H100, the dominant decode cost was a per-sequence recurrence loop that unrolled into roughly 30,000 near-empty kernel launches per step; batching it raised throughput 3.3×, and an integrated fused recurrent kernel 5.3×. (3) On four NVLink-connected H100s, four independent single-GPU replicas delivered 3.13× the aggregate throughput of 4-way tensor parallelism, which improved only 1.28× over one GPU. We also report an FP8 W8A8 Hopper MoE kernel that passes isolated correctness checks but has not been evaluated end to end, and we state the limits of every result, including the absence of an external-engine baseline.**

I. Introduction

Serving engines such as vLLM [1] and SGLang [2] organize per-sequence memory around a paged KV cache that grows with context length. Hybrid architectures break this assumption. A Gated DeltaNet (GDN) layer [3] keeps a constant-size matrix state per sequence and updates it in place at every token, while the full-attention layers interleaved with it still need a conventional KV cache. Qwen3.5-35B-A3B [4] combines a 3:1 interleave of the two with a 256-expert MoE [5], [6] in every layer, so per-sequence state, expert execution, and kernel scheduling all depart from the dense-transformer case.

Production engines such as vLLM support this model family, but inside large codebases where the cost of each design decision is hard to isolate. Minimal engines such as nano-vLLM [7] are small enough to reason about end to end, but support only KV-cache models. This work asks two questions: what does it take to serve a hybrid MoE model correctly in a minimal engine, and once it runs correctly, where does decode time actually go? Our contributions:

- A hybrid state design: a fixed-slot recurrent-state pool managed next to the paged KV cache at the same scheduler lifecycle points, compatible with CUDA-graph capture (Sec. III).
- A correctness-first methodology with numerical and behavioral gates, including three measurement artifacts that would otherwise have been reported as regressions (Sec. IV).
- A measured account of three optimization stages (W8A16 MoE quantization, batched hybrid decode, multi-GPU topology) that separates gains from our own code from gains from integrated third-party kernels (Sec. V, Table II).
- An explicit list of what these results do not show (Sec. VII).

TABLE I
Qwen3.5-35B-A3B, as served (values from the checkpoint config).

Parameters	34.66 B total; 94.7% in routed experts
Layers	40 = 10 blocks of [3 GDN + 1 GQA]
MoE (every layer)	256 experts, top-8 routed + 1 shared
GQA layers	16 query heads, 2 KV heads, dim 256
GDN layers	32 heads, d_k=d_v=128, fp32 state
KV cache per token	≈20 KB (10 layers, BF16)
GDN state per sequence	≈63 MB, independent of length

II. Background: the Model and Its Decode Step

Table I summarizes the model. A GDN layer applies the gated delta rule [3], [8] once per token:

$$S_t = \alpha_t\, S_{t-1}\left(I - \beta_t\, k_t k_t^\top\right) + \beta_t\, v_t k_t^\top, \qquad o_t = S_t\, q_t, \quad (1)$$

where $S_t \in \mathbb{R}^{d_v \times d_k}$ is a per-head state, α_t a decay gate and β_t a write strength. During prefill the recurrence can be evaluated in parallel chunks; during decode each sequence advances by exactly one token. The two memory types scale differently: the GDN state costs about 63 MB per sequence regardless of length, while the KV cache costs about 20 KB per token, so the two are equal near 3,100 tokens. Decode is memory-bound: each step reads the attention and GDN weights plus up to $8 \cdot B$ distinct experts for a batch of B sequences.

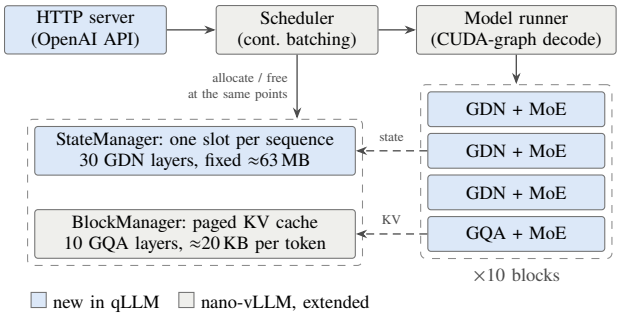


Fig. 1. qLLM architecture. The scheduler gives each admitted sequence both a KV block list and a recurrent-state slot, and frees both at completion or preemption.

TABLE II
PROVENANCE OF EACH COMPONENT. "INTEGRATED" MEANS THIRD-PARTY CODE CALLED AS A LIBRARY.

Component	Status
StateManager + scheduler hooks	Implemented
Hybrid model, TP sharding	Implemented
Batched GDN decode path	Implemented
W8A16 quantization pass	Implemented
Fused MoE Triton kernel	Adapted from vLLM
MoE alignment op (graph-safe)	Reimplemented
GDN kernels (fla [9])	Integrated
Attention (flash-attn [10])	Integrated
FP8 W8A8 Hopper MoE kernel	Developed, debugged
Scheduler, block mgr., TP runtime	nano-vLLM, extended

III. System Design

qLLM keeps nano-vLLM's scheduler, paged block manager, tensor-parallel (TP) runtime, and CUDA-graph decode, and adds three components (Fig. 1).

StateManager. A pool with one slot per concurrently running sequence holds each sequence's GDN recurrent state and short-convolution buffer for all 30 GDN layers. The scheduler allocates a slot at admission, at the same point the block manager allocates KV blocks, and frees it at completion or preemption. Slots are zeroed on allocation so a reused slot cannot leak state between requests. Because a captured CUDA graph has a fixed batch size, the pool reserves one extra scratch slot that padded graph entries write to; slot indices are passed to the model as a tensor, so state gathers and scatters have fixed shapes and can be captured.

Hybrid model. The model definition implements GDN, GQA and MoE layers, shards GDN heads and distributes experts across TP ranks, and replicates the two GQA KV heads when the TP degree exceeds two.

Server. An OpenAI-compatible HTTP server, with one background thread driving the scheduler, so throughput is always measured through the real request path.

IV. Methodology

Numerical gate. An unmodified PyTorch reference implementation, itself checked against Hugging Face (logit cosine 0.9972–0.9998), is the single source of truth. Components must reach cosine similarity > 0.99 against it; the reference's own agreement with Hugging Face is why a threshold of 1.0 would be unrealistic.

TABLE III
CORRECTNESS EVIDENCE. EACH ROW IS A SEPARATE RUN AND NAMES THE CONFIGURATION IT COVERS. ALL GSM8K ROWS USE W8A16 EXPERTS EXCEPT THE BF16 ARM OF THE PAIRED TEST.

Check / configuration	HW	Result
Checkpoint vs. HF (5 prompts)	–	cos 0.9982–0.9997
Eager vs. CUDA-graph decode	–	cos 1.000000
W8A16 expert layer vs. BF16	–	cos 0.999978
Fused MoE kernel vs. reference	–	cos 0.999988
GSM8K paired n=40, BF16 vs. W8A16	A6000	40/40 vs. 40/40
GSM8K all 1,319	A6000	95.30%
GSM8K all 1,319, sequential GDN	H100	95.91%
GSM8K n=100, batched GDN decode	H100	98.0%
GSM8K n=200, fla GDN decode	H100	97.0%
GSM8K n=60, fla, conc. 1 vs. 32	H100	59/60 vs. 59/60
Any 4-GPU configuration	4×H100	not run

Behavioral gate. Numerical checks miss errors that compound over hundreds of decode steps, so every change that touches the decode path is also checked on GSM8K [11] (greedy decoding, chat template with thinking disabled, pass bar 87.5%). Paired configurations are compared with McNemar's test [12]. Table III lists every gate with the exact configuration it covers.

Three measurement artifacts. (i) With a raw prompt format, 28% of answers could not be extracted at all; the chat format cut this to 1.6%. (ii) Under the raw format, an apparent quantization regression (25/40 vs. 20/40 correct, $p = 0.0625$) turned out to be a formatting and scoring artifact: after the fix, both arms scored 40/40 with zero discordant pairs. (iii) A 15-point eager-vs-CUDA-graph gap at $n = 40$ fell to $p = 0.38$ at $n = 100$. Each would have sent us chasing a bug that did not exist.

Throughput protocol. An HTTP client sends synthetic 128-token prompts at a fixed concurrency (requests in flight) and forces a fixed output length by ignoring end-of-sequence. Throughput is generated tokens divided by wall-clock time, so prefill is included. We run one warm-up and three trials and report the mean; trial spread is ≤2% except for the fla decode path (≈40 tok/s). We compare numbers only within one machine and software stack: the same unoptimized engine measured 206 and 277 tok/s on two H100 machines whose software stacks differed.

Hardware. 2×A6000 (48 GB, no NVLink); single H100s on two rented machines (one PCIe); and H100s on Modal, singly and as 4×H100 whose NVLink we verified directly (18 active links per GPU).

V. Results

A. W8A16 MoE quantization (2×A6000, TP=2)

In BF16 the weights occupy 35.0 GB of each 48 GB GPU, and concurrency 32 runs out of memory once CUDA-graph pools and expert-gather buffers are allocated. Because 94.7%

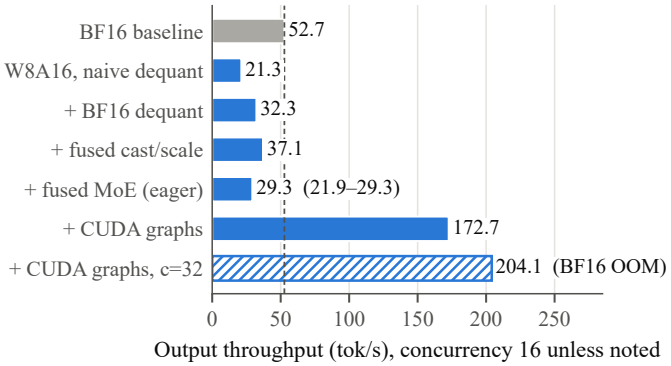


Fig. 2. Decode throughput through the W8A16 work (2×A6000, TP=2, 128-token prompts, 1,024 output tokens). Smaller weights first made decoding slower; the gain came from fusion and launch elimination. The hatched bar runs at a concurrency BF16 cannot reach.

TABLE IV
BF16 vs. W8A16 EXPERTS, 2×A6000, TP=2.

	BF16	W8A16
Weights per GPU	35.0 GB	19.06 GB
Throughput, concurrency 16	52.7–54.0	172.7
Throughput, concurrency 32	OOM	204.1
GSM8K paired, n=40	40/40	40/40
GSM8K full 1,319	not run	95.30%

of parameters are routed experts, we quantize only those: symmetric round-to-nearest INT8 with one scale per group of 128 input channels, with no calibration data. Weight memory falls to 19.06 GB per GPU (1.84×); the reconstruction error is 0.46–0.47% and a quantized layer matches BF16 at cosine 0.999978. Activations stay in BF16, so this is weight-only W8A16.

Fig. 2 shows the sequence. The first implementation gathered the selected experts' INT8 weights, dequantized them through FP32 and cast back to BF16, moving more bytes than it saved: 21.3 tok/s against 52.7 for BF16. Dequantizing directly to BF16 and fusing the cast with the scale recovered 74% of the loss (37.1 tok/s). We then adapted vLLM's fused-MoE Triton kernel [1], [13] to per-group scales. It reads INT8 weights in place and dequantizes in registers, so no gathered weight buffer is ever materialized. In isolation it was 4.71× faster than our pipeline (cosine 0.999988), yet inside the engine eager mode was *slower* (21.9–29.3 tok/s): the token-sorting and alignment step before the kernel spent about 57% of MoE time on per-operation dispatch. We reimplemented that step in PyTorch with fixed worst-case output shapes, which made the whole MoE path capturable in a CUDA graph. The result was 172.7 tok/s at concurrency 16, and 204.1 tok/s at concurrency 32, where BF16 runs out of memory.

Attribution caveat. The matched-concurrency gain is 3.2× (Table IV), but it should not be credited to quantization alone. The BF16 baseline used the original gather-then-matmul MoE path, and we never paired BF16 with a fused kernel. What quantization demonstrably provided is capacity (concurrency

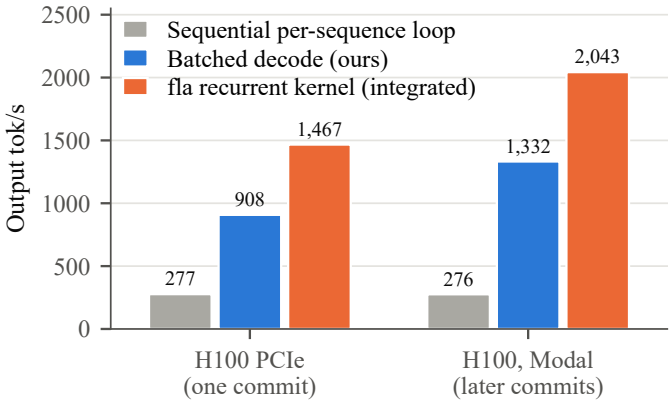


Fig. 3. Single-H100 decode throughput by GDN decode path (concurrency 64, 128-token prompts, 1,024 output tokens). Left: one machine and one commit. Right: a later Modal session after compiling the remaining decode glue; its sequential bar comes from an earlier commit on the same platform.

32) and a halved weight read for the fused kernel; the rest is fusion and launch elimination.

B. Hybrid decode (single H100, concurrency 64)

On the first H100 the engine reached 206 tok/s, barely above the 2×A6000 result, and gained only 7–9% from concurrency 32 to 64. Both facts pointed away from memory bandwidth. In decode mode, each GDN layer looped over the sequences in Python, issuing about 15 small operations per sequence; at concurrency 64 that is $64 \times 30 = 1{,}920$ iterations per step. CUDA-graph capture unrolls the loop, so each replayed step contained roughly 30,000 near-empty kernels, at a cost that grows linearly with the batch.

Batched decode (ours). In decode each sequence contributes exactly one token and the recurrences are independent, so we vectorized the loop body over a leading batch dimension: gather the states by slot index, apply one recurrence step as batched tensor operations on (B, H, d_k, d_v) tensors, and scatter the states back. With no data-dependent shapes, the path is capturable by construction. It is bitwise-identical to the loop at B=1 and within 10^{-8} at B=64 (checked on CPU), and raised throughput 3.3× (Fig. 3).

Fused recurrent kernel (integrated). fla's fused recurrent gated-delta-rule kernel [9] performs the same step in one Triton kernel per layer. It captures cleanly in a CUDA graph (fla's chunked prefill kernel does not, so prefill runs outside graphs) and gives 5.3×. It reorders floating-point summation, so it is not bitwise-identical; GSM8K stayed at 97.0% (n=200), and identical scores at concurrency 1 and 32 show no cross-sequence contamination. Scaling from concurrency 32 to 64 recovered: +26% (batched) and +46% (fla), against +7% before.

A later session compiled the remaining decode-only glue code and fixed a recompilation-cache limit, reaching 1,332 (batched) and 2,043 tok/s (fla) on a Modal H100. The best number is 7.4× the sequential path measured with the same harness on that platform. A profile of the batched path (Fig. 4) shows GDN still taking 61% of GPU time: 46% for unfused

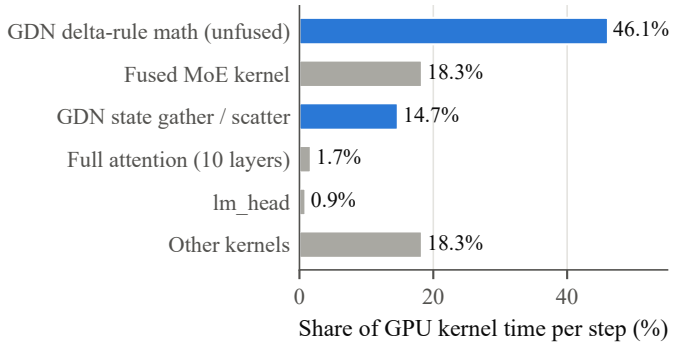


Fig. 4. GPU kernel-time share of one decode step on the batched GDN path (nsys with per-node CUDA-graph tracing, H100, concurrency 64). Blue rows are GDN work.

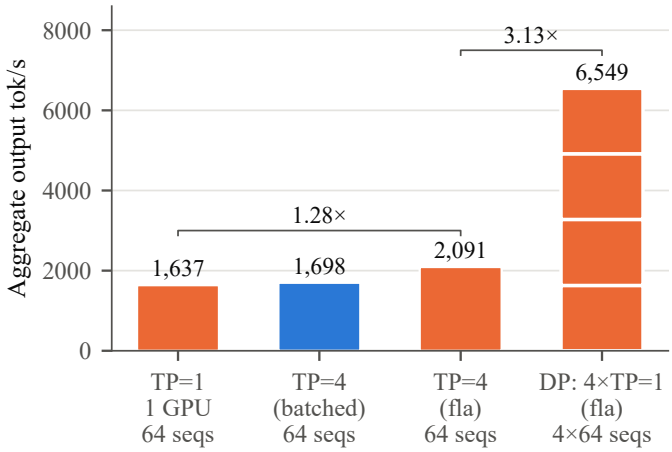


Fig. 5. Aggregate throughput on 4×H100 with NVLink (128-token prompts, 256 output tokens). The DP bar stacks four independent replicas run simultaneously; the TP=1 bar is their mean. Sequences in flight are listed under each bar.

delta-rule arithmetic over fp32 temporaries and 15% for state gathers and scatters. With the fla kernel that arithmetic disappears and the MoE kernel becomes the largest single kernel (10.4%).

A null result. Retuning the MoE kernel's tile sizes for the decode shape (each expert sees about two tokens at concurrency 64) made the isolated MoE pipeline 2.49× faster at unchanged accuracy (cosine 0.999988), but end-to-end throughput did not improve (1,942 vs. a 1,962–2,043 range), so we reverted it. MoE arithmetic is not what bounds the step.

C. Multi-GPU topology (4×H100, NVLink)

We compared two ways to use four GPUs at 256 output tokens (Fig. 5). With 4-way TP the engine reached 1,698 tok/s (batched) and 2,091 tok/s (fla), only 1.28× a single TP=1 engine under the same settings. The likely reason is that sharding shrinks each kernel's work but not the number of kernels: every rank still issues the full launch sequence per step, plus all-reduces. Running four *independent* TP=1 replicas, one per GPU with no inter-GPU communication, reached 6,549 tok/s, 3.13× the TP=4 result; the replicas agreed within 1.1%. At 1,024 output tokens the replicas reached 7,858 tok/s, 96% of four times the best single-GPU result. Each replica holds a full W8A16 copy of the model, which fits in 80 GB.

This comparison has a caveat that matters. Each engine is capped at 64 concurrent sequences in our deployment, so the four replicas carried 4×64 sequences while the single TP=4 engine carried 64. TP=4 at a larger batch, which could amortize its fixed costs better, was not measured, and we measured throughput only, not per-request latency, where TP could win.

D. An FP8 W8A8 Hopper MoE kernel (isolated)

W8A16 still multiplies in BF16. To use Hopper's FP8 tensor cores [14] for both operands, we developed a kernel that fuses the gate/up GEMM, SwiGLU, dynamic FP8 requantization of the intermediate, the down GEMM and the weighted expert combine, using `wgmma` and TMA instructions (`sm_90a`). Weights use 128×128 block scales, as in DeepSeek-V3 [15], and activations use one scale per token per 128 channels, computed dynamically without calibration.

TABLE V
ROOT CAUSES FOUND IN THE FP8 KERNEL.

Symptom	Root cause
~2×10^5 blow-up on some tokens	Activation scales computed per token, but read per (token, 128-channel block): out-of-bounds reads
Wrong up-projection scale	SwiGLU weight scale chosen by register pair instead of by warpgroup
Plausible magnitudes, cosine ≈ 0	Kernel pairs accumulator rows r and r+8 as (gate, up); checkpoints store contiguous halves, so weights must be interleaved at load

The first smoke test produced cosine 0.01 against a PyTorch reference. We debugged it by isolation: we chose a single token, output feature and K-block, computed the expected value by hand at each stage, and compared it with the kernel. This found three independent root causes (Table V). After the fixes, cosine is 0.998104 and `compute-sanitizer` reports no memory errors.

The validated scope is narrow: synthetic weights, 8 experts, hidden size 256, 16 tokens, and one tile configuration. The load-time interleaving for real checkpoints is implemented and CPU-tested but has not run on a GPU. We have no end-to-end throughput or accuracy for this kernel; all served results in this paper are W8A16.

VI. Discussion

Decode here is bound by kernel count, not bandwidth. Four independent observations agree: the sequential GDN loop's cost grew with the number of launches; a 2.49× faster MoE kernel left end-to-end throughput unchanged; TP=4 gained only 1.28×; and replicas that each pay the launch cost on their own GPU scaled almost linearly. Even the fla-path profile still shows about 885 tiny copy kernels per step. The next lever is fewer kernels per step (fusing the GDN glue and state I/O, BF16 state), not faster matrix multiplication.

Bytes saved count only if the kernel does not re-expand them. Weight-only quantization first regressed throughput because dequantization materialized wider tensors. True W8A8 adds a per-token activation-quantization step to the hot path, and its cost has to be measured against the bandwidth it saves, which we have not yet done. Activation outliers are the known difficulty here [16], [17]; per-block activation scales confine an outlier's effect to 128 channels, but we have not measured their effect on accuracy.

Isolated wins do not transfer automatically. A 4.71× isolated MoE speedup was slower in the eager engine, and a 2.49× retune did nothing end to end. Only the real engine, with graph capture and neighboring kernels, settles a performance claim.

VII. Limitations

- **No external baseline.** vLLM supports this model, but we did not benchmark it or SGLang. Every speedup here is relative to an earlier state of our own engine.
- **Weight-only quantization.** All served numbers use W8A16. The FP8 W8A8 kernel is validated in isolation only.
- **Thin accuracy evidence.** BF16 and W8A16 were compared on 40 paired GSM8K items only; there is no full-set BF16 run. GSM8K is a single, coarse task (greedy decoding, no thinking, 512-token cap). The fla path has not had a full 1,319-item run, and the headline 2,043 and 6,549 tok/s configurations were not re-gated at their exact commit or topology.
- **Narrow workload.** Synthetic 128-token prompts, fixed output lengths, at most 64 sequences per engine, throughput only (no time-to-first-token or per-token latency). Prefix caching is disabled for the hybrid model, and prefill is not interleaved with decode.
- **Unverified TP=4 numerics.** Both 4-GPU configurations lack a correctness gate, and the TP=4 profile traces have not been analyzed.
- **Target not met.** The project target of 17–20k tok/s on 4×H100 was not reached; the best result is about 33–39% of it.

VIII. Conclusion

Serving a hybrid GDN/MoE model in a minimal engine needed one new memory abstraction, a fixed-slot recurrent-state pool managed alongside the KV cache. After that, most of the throughput gain came from removing kernel launches and intermediate tensors rather than from smaller weights: CUDA-graph capture of the MoE routing path, batching the GDN recurrence, and replication instead of sharding. The most useful next steps are an external baseline at matched precision, a full-set GSM8K gate on the fastest configuration, and an end-to-end evaluation of the FP8 W8A8 kernel.

References

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[2] L. Zheng *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[3] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated delta networks: Improving Mamba2 with delta rule," in *International Conference on Learning Representations (ICLR)*, 2025.

[4] Qwen Team, "Qwen3.5: Towards native multimodal agents," https://qwen.ai/blog?id=qwen3.5, Feb. 2026.

[5] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations (ICLR)*, 2017.

[6] D. Dai *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024.

[7] GeeeekExplorer, "nano-vLLM: A lightweight vLLM implementation," https://github.com/GeeeekExplorer/nano-vllm, 2025.

[8] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, "Parallelizing linear transformers with the delta rule over sequence length," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[9] S. Yang and Y. Zhang, "FLA: A Triton-based library for hardware-efficient implementations of linear attention mechanism," https://github.com/fla-org/flash-linear-attention, 2024.

[10] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[11] K. Cobbe *et al.*, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[12] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[13] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.

[14] P. Micikevicius *et al.*, "FP8 formats for deep learning," *arXiv preprint arXiv:2209.05433*, 2022.

[15] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[16] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[17] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning (ICML)*, 2023.